

Lesson 5 Notes: Lower Triangular Matrix Row-Major Mapping

1. Main idea of Lesson 5

Lesson 5 introduces a new type of special matrix: the **lower triangular matrix**.

Before this lesson, we studied **diagonal matrices**.

A diagonal matrix stores values only when:

$$i == j$$

That means only the main diagonal is stored.

In Lesson 5, the rule changes.

A lower triangular matrix stores values when:

$$i \geq j$$

That means we store values on the main diagonal and below the main diagonal.

So Lesson 5 teaches:

- what a lower triangular matrix is
- which positions are stored
- which positions are structural zeros
- how many values are stored
- what row-major storage means
- how to map a 2D matrix position (i, j) into a 1D array index
- how to calculate indexes by hand
- how to write a helper function for the formula
- what the time and space complexity are

2. What you already know before Lesson 5

Before learning Lesson 5, you should already understand the diagonal matrix idea.

A diagonal matrix stores only positions where the row number and column number are the same.

Example:

```
5  0  0  0
0  8  0  0
0  0  9  0
0  0  0 12
```

The stored values are:

```
5, 8, 9, 12
```

The compact array is:

```
A = [5, 8, 9, 12]
```

The diagonal condition is:

```
i == j
```

The diagonal index formula is:

```
index = i - 1
```

Lesson 5 uses the same general idea, but with a bigger stored region.

Instead of storing only one diagonal line, we store a whole triangle.

3. What is a lower triangular matrix?

A **lower triangular matrix** is a square matrix where all useful values are on or below the main diagonal.

The values above the main diagonal are zero.

Example:

```
1  0  0  0
2  3  0  0
```

```
4 5 6 0
7 8 9 10
```

The useful part is the lower-left triangle:

```
1
2 3
4 5 6
7 8 9 10
```

That is why it is called **lower triangular**.

The meaningful values form a triangle under the main diagonal.

4. Main diagonal reminder

The **main diagonal** goes from the top-left corner to the bottom-right corner.

Example:

```
1 0 0 0
0 3 0 0
0 0 6 0
0 0 0 10
```

The main diagonal positions are:

```
M[1,1]
M[2,2]
M[3,3]
M[4,4]
```

For any main diagonal position:

```
i == j
```

In a lower triangular matrix, we store the main diagonal too.

So all positions where `i == j` are stored.

But we also store positions below the diagonal.

5. Lower triangular condition

For a lower triangular matrix:

$$i \geq j$$

means the position is stored.

Here:

- **i** means row number
- **j** means column number

So if the row number is greater than or equal to the column number, the value belongs to the lower triangle.

Examples:

```
M[1,1] is stored because 1 >= 1
M[2,1] is stored because 2 >= 1
M[2,2] is stored because 2 >= 2
M[3,1] is stored because 3 >= 1
M[3,2] is stored because 3 >= 2
M[3,3] is stored because 3 >= 3
```

6. Zero region condition

For a lower triangular matrix:

$$i < j$$

means the position is above the main diagonal.

These positions are structural zeros.

Examples:

```
M[1,2] is zero because 1 < 2
M[1,3] is zero because 1 < 3
M[1,4] is zero because 1 < 4
M[2,3] is zero because 2 < 3
M[2,4] is zero because 2 < 4
M[3,4] is zero because 3 < 4
```

These zeros are not stored in the array.

The program simply returns or prints zero when needed.

7. Stored region vs zero region

For this matrix:

```
1  0  0  0
2  3  0  0
4  5  6  0
7  8  9  10
```

Stored positions are:

```
M[1,1]
M[2,1] M[2,2]
M[3,1] M[3,2] M[3,3]
M[4,1] M[4,2] M[4,3] M[4,4]
```

Zero positions are:

```
M[1,2] M[1,3] M[1,4]
M[2,3] M[2,4]
M[3,4]
```

The stored positions follow:

```
i >= j
```

The zero positions follow:

$$i < j$$

8. Comparison with diagonal matrix

The biggest change from diagonal matrix to lower triangular matrix is the condition.

Matrix type	Stored condition	Meaning
Diagonal matrix	$i == j$	Store only the main diagonal
Lower triangular matrix	$i \geq j$	Store the main diagonal and everything below it

So a lower triangular matrix stores more values than a diagonal matrix.

A diagonal matrix stores only one value per row.

A lower triangular matrix stores more values as we go down the rows.

9. What is a structural zero?

A **structural zero** is a zero that exists because of the matrix rule.

In a lower triangular matrix, every position above the main diagonal is a structural zero.

Example:

$$M[2,4] = 0$$

Why?

Because:

$$2 < 4$$

So $(2,4)$ is above the main diagonal.

It is not stored.

The program already knows it must be zero.

10. Why lower triangular storage saves memory

A normal matrix stores every value.

For an $N \times N$ matrix, a normal matrix stores:

N^2 values

But a lower triangular matrix stores only the lower triangle.

For an $N \times N$ lower triangular matrix, the number of stored values is:

$N(N + 1) / 2$

This is about half of the full matrix.

So lower triangular storage saves memory by not storing the zeros above the main diagonal.

11. Counting stored values

Look at a lower triangular matrix row by row.

For a 5×5 lower triangular matrix:

Row 1 stores 1 value
Row 2 stores 2 values
Row 3 stores 3 values
Row 4 stores 4 values
Row 5 stores 5 values

So the total stored values are:

$1 + 2 + 3 + 4 + 5 = 15$

The formula for this is:

$N(N + 1) / 2$

For $N = 5$:

$$\begin{aligned}
 &5(5 + 1) / 2 \\
 &= 5(6) / 2 \\
 &= 30 / 2 \\
 &= 15
 \end{aligned}$$

So a 5 x 5 lower triangular matrix stores 15 values.

12. Counting structural zeros

A full 5 x 5 matrix has:

$$5 \times 5 = 25 \text{ values}$$

A lower triangular 5 x 5 matrix stores:

$$15 \text{ values}$$

So the number of structural zeros is:

$$25 - 15 = 10$$

The general formula for structural zeros is:

$$N(N - 1) / 2$$

For $N = 5$:

$$\begin{aligned}
 &5(5 - 1) / 2 \\
 &= 5(4) / 2 \\
 &= 20 / 2 \\
 &= 10
 \end{aligned}$$

So there are 10 structural zeros.

13. Stored values formula

For an $N \times N$ lower triangular matrix:

$$\text{stored values} = N * (N + 1) / 2$$

This tells us how large the 1D array should be.

Example:

If $N = 4$:

$$\begin{aligned}\text{stored values} &= 4(4 + 1) / 2 \\ \text{stored values} &= 4(5) / 2 \\ \text{stored values} &= 20 / 2 \\ \text{stored values} &= 10\end{aligned}$$

So for a 4 x 4 lower triangular matrix, the 1D array needs 10 cells.

14. Zero values formula

For an $N \times N$ lower triangular matrix:

$$\text{zero values} = N * (N - 1) / 2$$

Example:

If $N = 4$:

$$\begin{aligned}\text{zero values} &= 4(4 - 1) / 2 \\ \text{zero values} &= 4(3) / 2 \\ \text{zero values} &= 12 / 2 \\ \text{zero values} &= 6\end{aligned}$$

A 4 x 4 full matrix has 16 positions.

The lower triangular matrix stores 10 positions.

The remaining 6 positions are structural zeros.

15. What does row-major mean?

Row-major means we store values row by row.

We start from row 1.

Then we store row 2.

Then row 3.

Then row 4.

And so on.

For a lower triangular matrix, each row stores only the values that belong to the lower triangle.

Example:

```
1  0  0  0
2  3  0  0
4  5  6  0
7  8  9  10
```

Row-major storage means:

```
Row 1: 1
Row 2: 2 3
Row 3: 4 5 6
Row 4: 7 8 9 10
```

So the 1D array becomes:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

16. Visual row-major mapping

Using this matrix:

```
1  0  0  0
2  3  0  0
4  5  6  0
7  8  9  10
```

The 1D array is:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Mapping:

```
M[1,1] -> A[0]
```

```
M[2,1] -> A[1]
```

```
M[2,2] -> A[2]
```

```
M[3,1] -> A[3]
```

```
M[3,2] -> A[4]
```

```
M[3,3] -> A[5]
```

```
M[4,1] -> A[6]
```

```
M[4,2] -> A[7]
```

```
M[4,3] -> A[8]
```

```
M[4,4] -> A[9]
```

This shows that the lower triangle is flattened into one row.

17. Matrix indexing vs array indexing

In these lessons, matrix positions use **1-based indexing**.

That means matrix rows and columns start at 1.

Examples:

```
M[1,1]
```

```
M[2,1]
```

```
M[3,2]
```

But C and C++ arrays use **0-based indexing**.

That means array indexes start at 0.

Examples:

```
A[0]
A[1]
A[2]
```

This is why the formula uses:

```
j - 1
```

The `-1` helps convert from 1-based matrix position to 0-based array index.

18. The row-major index formula

For lower triangular row-major storage, the index formula is:

```
index = i * (i - 1) / 2 + (j - 1)
```

This formula should be used only when:

```
i >= j
```

If:

```
i < j
```

then the position is not stored.

It is a structural zero.

19. Meaning of the formula

The formula is:

```
index = i * (i - 1) / 2 + (j - 1)
```

It has two main parts.

Part 1

```
i * (i - 1) / 2
```

This counts how many stored values come before row `i`.

In simple words:

```
Skip all previous rows.
```

Part 2

```
j - 1
```

This moves inside the current row to column `j`.

In simple words:

```
Move to the correct place inside the current row.
```

So the full meaning is:

```
index = values skipped from previous rows + movement inside the current row
```

20. Why `i * (i - 1) / 2` skips previous rows

Suppose we want a value in row `i`.

Before row `i`, there are rows:

```
1, 2, 3, ..., i - 1
```

In a lower triangular matrix:

```
Row 1 stores 1 value  
Row 2 stores 2 values  
Row 3 stores 3 values
```

...

Row $i - 1$ stores $i - 1$ values

So values before row i are:

$$1 + 2 + 3 + \dots + (i - 1)$$

The formula for this sum is:

$$i * (i - 1) / 2$$

That is why this part skips all previous rows.

21. Why $j - 1$ moves inside the current row

Inside row i , the stored columns are:

1, 2, 3, ..., i

If the target is column j , then we move forward by:

$$j - 1$$

Examples:

Column 1 means move 0 steps: $j - 1 = 1 - 1 = 0$
Column 2 means move 1 step: $j - 1 = 2 - 1 = 1$
Column 3 means move 2 steps: $j - 1 = 3 - 1 = 2$
Column 4 means move 3 steps: $j - 1 = 4 - 1 = 3$

This is why the second part of the formula is $j - 1$.

22. Formula in plain English

The formula:

```
index = i * (i - 1) / 2 + (j - 1)
```

means:

First, skip all rows above row i .
Then, move to column j inside row i .

That is the easiest way to remember it.

23. Dry run 1: Find the index of $M[4,3]$

We want to find the array index for:

```
M[4,3]
```

Here:

```
i = 4  
j = 3
```

Step 1: Check the condition

```
4 >= 3
```

This is true.

So `M[4,3]` is stored.

Step 2: Skip previous rows

Rows before row 4 are:

```
Row 1, Row 2, Row 3
```

They store:

```
1 + 2 + 3 = 6 values
```

So we skip 6 values.

Step 3: Move inside row 4

The target column is 3.

So movement inside the row is:

$$j - 1 = 3 - 1 = 2$$

Step 4: Add both parts

```
index = 6 + 2  
index = 8
```

So:

$M[4,3] \rightarrow A[8]$

Using the formula

$$\text{index} = i * (i - 1) / 2 + (j - 1)$$

Substitute values:

```
index = 4 * (4 - 1) / 2 + (3 - 1)  
index = 4 * 3 / 2 + 2  
index = 12 / 2 + 2  
index = 6 + 2  
index = 8
```

Final answer:

$M[4,3]$ is stored at $A[8]$

24. Dry run 2: Find the index of $M[5,4]$

We want to find the array index for:

$M[5,4]$

Here:

$i = 5$
 $j = 4$

Step 1: Check the condition

$5 \geq 4$

This is true.

So $M[5,4]$ is stored.

Step 2: Skip previous rows

Rows before row 5 are:

Row 1, Row 2, Row 3, Row 4

They store:

$1 + 2 + 3 + 4 = 10$ values

So we skip 10 values.

Step 3: Move inside row 5

The target column is 4.

So movement inside row 5 is:

$j - 1 = 4 - 1 = 3$

Step 4: Add both parts

```
index = 10 + 3  
index = 13
```

So:

```
M[5,4] -> A[13]
```

Using the formula

```
index = i * (i - 1) / 2 + (j - 1)
```

Substitute values:

```
index = 5 * (5 - 1) / 2 + (4 - 1)  
index = 5 * 4 / 2 + 3  
index = 20 / 2 + 3  
index = 10 + 3  
index = 13
```

Final answer:

```
M[5,4] is stored at A[13]
```

25. Dry run 3: Find the index of M[5,2]

We want:

```
M[5,2]
```

Here:

```
i = 5  
j = 2
```

Check the condition:

```
5 >= 2
```

This is true.

So the position is stored.

Formula:

```
index = i * (i - 1) / 2 + (j - 1)
```

Substitute:

```
index = 5 * (5 - 1) / 2 + (2 - 1)
index = 5 * 4 / 2 + 1
index = 20 / 2 + 1
index = 10 + 1
index = 11
```

So:

```
M[5,2] -> A[11]
```

26. Dry run 4: Check M[2,5]

We want to check:

```
M[2,5]
```

Here:

```
i = 2
j = 5
```

Check the condition:

```
2 >= 5
```

This is false.

So `M[2,5]` is not stored.

Since:

$$2 < 5$$

this position is above the main diagonal.

Therefore:

$$M[2,5] = 0$$

It is a structural zero.

27. Helper function for row-major index

A helper function can calculate the array index for a lower triangular matrix.

Code:

```
int getArrayIndex(int i, int j) {  
    if (i >= j) {  
        return (i * (i - 1) / 2) + (j - 1);  
    }  
    return -1;  
}
```

This function returns:

- the correct index if the position is stored
 - `-1` if the position is a structural zero
-

28. Why return -1?

The function returns `-1` when the position is not stored.

Example:

```
getArrayIndex(2, 5)
```

Since:

```
2 < 5
```

this position is above the main diagonal.

So the function returns:

```
-1
```

Why `-1`?

Because valid array indexes start from 0.

Valid indexes are:

```
0, 1, 2, 3, ...
```

So `-1` is a useful signal meaning:

```
This matrix position is not stored.
```

29. Helper function examples

Example 1

```
getArrayIndex(4, 3)
```

Check:

```
4 >= 3
```

True.

Calculate:

```
index = 4 * (4 - 1) / 2 + (3 - 1)
index = 4 * 3 / 2 + 2
index = 6 + 2
index = 8
```

Return:

8

Example 2

```
getArrayIndex(5, 4)
```

Check:

5 >= 4

True.

Calculate:

```
index = 5 * (5 - 1) / 2 + (4 - 1)
index = 10 + 3
index = 13
```

Return:

13

Example 3

```
getArrayIndex(2, 5)
```

Check:

2 >= 5

False.

Return:

-1

30. Set idea for lower triangular matrix

A `set` operation stores a value only if the position belongs to the lower triangle.

Rule:

```
if (i >= j)
```

then store the value.

If:

```
i < j
```

then ignore the value.

Basic idea:

```
void set(int A[], int i, int j, int x) {  
    if (i >= j) {  
        int index = i * (i - 1) / 2 + (j - 1);  
        A[index] = x;  
    }  
}
```

This protects the lower triangular structure.

It prevents storing values above the main diagonal.

31. Get idea for lower triangular matrix

A `get` operation returns a stored value if the position belongs to the lower triangle.

Rule:

```
if (i >= j)
```

then calculate the index and return the stored value.

If:

```
i < j
```

then return zero.

Basic idea:

```
int get(int A[], int i, int j) {  
    if (i >= j) {  
        int index = i * (i - 1) / 2 + (j - 1);  
        return A[index];  
    }  
    return 0;  
}
```

This works because positions above the main diagonal are structural zeros.

32. Display idea for lower triangular matrix

To display the full matrix, we still use nested loops.

The stored array only contains the lower triangle.

But the display must show the full square matrix.

So for every position `(i, j)`:

- if `i >= j`, print the stored value
- if `i < j`, print `0`

Basic idea:

```
void display(int A[], int n) {  
    for (int i = 1; i <= n; i++) {  
        for (int j = 1; j <= n; j++) {  
            if (i >= j) {
```



```

        int index = i * (i - 1) / 2 + (j - 1);
        cout << A[index] << " ";
    } else {
        cout << "0 ";
    }
}
cout << endl;
}
}

```

Display still prints every visible position.

So display takes $O(N^2)$ time.

33. Stored array vs visible matrix

The stored array is what the computer actually keeps in memory.

The visible matrix is what the user sees.

Example visible matrix:

```

1  0  0  0
2  3  0  0
4  5  6  0
7  8  9  10

```

Stored array:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

The zeros are not stored.

They are printed when needed.

34. Beginner mental model

Think of the full matrix as what the user sees.

Think of the 1D array as what the computer stores.

For lower triangular row-major storage:

The computer reads the matrix row by row.
It stores only the values on or below the diagonal.
It skips the zeros above the diagonal.

So the full matrix:

```
1  0  0  0
2  3  0  0
4  5  6  0
7  8  9  10
```

becomes:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

The array is smaller, but we can still rebuild the full matrix view.

35. The most important thinking pattern

For any lower triangular row-major index problem, ask two questions:

Question 1

How many stored values come before this row?

Answer:

```
i * (i - 1) / 2
```

Question 2

How far do I move inside the current row?

Answer:

$$j - 1$$

Then add both answers:

$$\text{index} = i * (i - 1) / 2 + (j - 1)$$

This is the core skill of Lesson 5.

36. Complexity of lower triangular row-major mapping

Storage complexity

A lower triangular matrix stores:

$$N * (N + 1) / 2$$

This is about half of a full $N \times N$ matrix.

In Big-O notation, this is still:

$$O(N^2)$$

Why?

Because the number of stored values still grows like N^2 , even though it is only about half of N^2 .

So the storage is:

$$O(N^2), \text{ but optimized to about half of full storage}$$

Index calculation complexity

The formula:

$$\text{index} = i * (i - 1) / 2 + (j - 1)$$

uses only arithmetic.

It does not use loops.

So index calculation is:

$O(1)$

Set complexity

The `set` operation checks one condition and calculates one formula.

So:

`set() =  $O(1)$`

Get complexity

The `get` operation checks one condition and calculates one formula if needed.

So:

`get() =  $O(1)$`

Display complexity

Display prints every position in the full matrix.

For an `N x N` matrix, there are:

N^2 visible positions

So:

`display() =  $O(N^2)$`

37. Complexity table

Operation	Time complexity	Space complexity	Reason
Storage	-	$O(N^2)$, about half of full storage	Stores $N(N + 1) / 2$ values
Index calculation	$O(1)$	-	Uses direct arithmetic
set()	$O(1)$	-	One condition and one index formula
get()	$O(1)$	-	One condition and one possible lookup
display()	$O(N^2)$	-	Prints every visible matrix position

38. Important comparison: diagonal vs lower triangular

Concept	Diagonal matrix	Lower triangular matrix
Stored condition	$i == j$	$i \geq j$
Zero condition	$i \neq j$	$i < j$
Stored shape	Main diagonal only	Main diagonal and below
Stored values	N	$N(N + 1) / 2$
Index formula	$i - 1$	$i(i - 1)/2 + (j - 1)$
Storage order	Diagonal values	Row by row lower triangle
Example zero	$M[2, 3]$	$M[2, 3]$

The big lesson is that the structure changes, so the condition and index formula also change.

39. Common beginner mistakes

Mistake 1: Using the diagonal condition

Wrong:

```
if (i == j)
```

Correct for lower triangular:

```
if (i >= j)
```

Reason:

A lower triangular matrix stores more than the main diagonal.

It stores everything below the diagonal too.

Mistake 2: Storing values above the diagonal

Wrong idea:

```
M[2,4] should be stored.
```

Correct idea:

```
M[2,4] is above the diagonal because 2 < 4.  
So it is a structural zero.
```

Mistake 3: Forgetting `j - 1`

Wrong:

```
index = i * (i - 1) / 2 + j
```

Correct:

```
index = i * (i - 1) / 2 + (j - 1)
```

Reason:

Matrix positions start at 1, but array indexes start at 0.

Mistake 4: Thinking storage is $O(N)$

Wrong idea:

Lower triangular storage is $O(N)$.

Correct idea:

Lower triangular storage is $O(N^2)$, but it uses about half the full matrix storage.

Reason:

It stores:

$$N(N + 1) / 2$$

This still grows like N^2 .

Mistake 5: Confusing row-major with column-major

Row-major means row by row.

For lower triangular row-major storage:

Row 1 first
Row 2 next
Row 3 next
...

Column-major storage is different and uses a different formula.

Lesson 5 is specifically about **row-major** mapping.

40. Practice questions

Question 1

Is $M[3, 2]$ stored?

Check:

$3 \geq 2$

Answer:

Yes, it is stored.

Question 2

Is $M[2,4]$ stored?

Check:

$2 \geq 4$

This is false.

Answer:

No, it is a structural zero.

Question 3

Find the index of $M[4,2]$.

Formula:

$$\text{index} = i * (i - 1) / 2 + (j - 1)$$

Substitute:

$$\begin{aligned}\text{index} &= 4 * (4 - 1) / 2 + (2 - 1) \\ \text{index} &= 4 * 3 / 2 + 1 \\ \text{index} &= 6 + 1 \\ \text{index} &= 7\end{aligned}$$

Answer:

M[4,2] -> A[7]

Question 4

Find the index of M[5,5] .

Check:

5 >= 5

True.

Formula:

```
index = 5 * (5 - 1) / 2 + (5 - 1)
index = 5 * 4 / 2 + 4
index = 10 + 4
index = 14
```

Answer:

M[5,5] -> A[14]

Question 5

What does `getArrayIndex(1,4)` return?

Check:

1 >= 4

False.

Answer:

-1

Reason:

`M[1,4]` is above the diagonal, so it is not stored.

41. Exit questions

You should be able to answer these before moving to Lesson 6.

1. What is a lower triangular matrix?
 2. What condition tells us a position is stored?
 3. What condition tells us a position is a structural zero?
 4. Why is `M[2,5]` not stored?
 5. How many values are stored in an `N x N` lower triangular matrix?
 6. How many values are stored when `N = 5`?
 7. What does row-major mean?
 8. What is the row-major index formula?
 9. What does the part `i * (i - 1) / 2` mean?
 10. What does the part `j - 1` mean?
 11. What index does `M[4,3]` map to?
 12. What index does `M[5,4]` map to?
 13. Why does the helper function return `-1` for positions above the diagonal?
 14. What is the storage complexity?
 15. What is the index calculation complexity?
-

42. Short answers to exit questions

1. A lower triangular matrix stores values on and below the main diagonal.
2. A position is stored when `i >= j`.
3. A position is a structural zero when `i < j`.
4. `M[2,5]` is not stored because `2 < 5`, so it is above the main diagonal.
5. It stores `N(N + 1) / 2` values.
6. When `N = 5`, it stores `15` values.
7. Row-major means storing values row by row.
8. The formula is `i * (i - 1) / 2 + (j - 1)`.
9. `i * (i - 1) / 2` counts stored values before row `i`.
10. `j - 1` moves inside the current row.
11. `M[4,3]` maps to `A[8]`.
12. `M[5,4]` maps to `A[13]`.
13. The function returns `-1` because the position is not stored, and `-1` is not a valid array index.
14. Storage complexity is `O(N2)`, but it uses about half of full matrix storage.

15. Index calculation complexity is $O(1)$.

43. Lesson 5 key rules

Lower triangular stored condition

$i \geq j$

Lower triangular zero condition

$i < j$

Stored values formula

$N * (N + 1) / 2$

Structural zeros formula

$N * (N - 1) / 2$

Row-major index formula

$\text{index} = i * (i - 1) / 2 + (j - 1)$

Helper function

```
int getArrayIndex(int i, int j) {  
    if (i >= j) {  
        return (i * (i - 1) / 2) + (j - 1);  
    }  
    return -1;  
}
```

44. Lesson 5 final summary

Lesson 5 teaches lower triangular matrix row-major mapping.

A lower triangular matrix stores values on and below the main diagonal.

The stored condition is:

```
i >= j
```

The zero condition is:

```
i < j
```

A lower triangular matrix does not store the zeros above the main diagonal.

For an `N x N` matrix, it stores:

```
N * (N + 1) / 2
```

values.

Row-major means the values are stored row by row.

The row-major index formula is:

```
index = i * (i - 1) / 2 + (j - 1)
```

This formula means:

```
Skip all previous rows, then move inside the current row.
```

The most important idea is not just memorizing the formula.

The most important idea is understanding why the formula works:

```
i * (i - 1) / 2  -> skip previous rows  
j - 1           -> move inside the current row
```

Once you understand that, lower triangular row-major mapping becomes much easier.